

DETECTION ENGINEERING & THREAT HUNTING PORTFOLIO

TEMP.Hex

Threat Hunting Lab — Case Study Compendium

SOGU / TEMP.Hex-inspired adversary emulation, Sysmon telemetry validation & ATT&CK-mapped detection engineering

Platform: **Windows 11**

Telemetry: **Sysmon (SwiftOnSecurity config)**

Status: **Campaign Complete**

LAB ENVIRONMENT SUMMARY

Operating System	Windows 11, single-host lab VM
Telemetry Source	Sysmon, installed with the SwiftOnSecurity configuration
Investigation Tooling	Windows Event Viewer / Get-WinEvent, manual registry & log review
Campaign Inspiration	TEMP.Hex / SOGU tradecraft (USB delivery, staging, beaconing, persistence, obfuscation)
Lifecycle Simulated	Initial Access → Discovery → Staging → C2 / Beaconing → Persistence → Obfuscated Execution

T1091 · Removable Media

T1059.001 · PowerShell

T1059.003 · Cmd Shell

T1082 · System Info Disc.

T1033 · Owner/User Disc.

T1074.001 · Local Staging

T1071.001 · Web Protocols

T1547.001 · Registry Run Keys

T1036 · Masquerading

T1105 · Ingress Tool Transfer

T1027 · Obfuscated Files

T1140 · Deobfuscate/Decode

⚠️ **LAB ISOLATION & SAFETY DISCLAIMER**

All activity described in this document was performed inside an isolated, non-production Windows 11 lab virtual machine built specifically for this project. Every "payload" used is benign — native Windows binaries (notepad.exe), first-party PowerShell cmdlets (Get-Date, Invoke-WebRequest), and requests to the reserved documentation domain example.com. No real malware, exploit code, unauthorized access, or third-party infrastructure was involved at any stage. The goal throughout was to generate realistic, TEMP.Hex-style telemetry in order to practice threat hunting and detection engineering, not to cause harm. This report is intended for professional portfolio and training review.

Reading Guide & Case Index

Environment

COMPONENT	VALUE
Operating System	Windows 11
Telemetry	Sysmon
Investigation Tool	Event Viewer
Campaign Inspiration	TEMP.Hex / SOGU

Hunting Areas Covered

- **Initial Access** — execution from a simulated removable-media staging path (T1091)
- **Discovery & Staging** — host recon and local data staging feeding PowerShell/payload execution (T1059, T1082, T1033, T1074.001)
- **Command & Control** — fixed-interval PowerShell beaconing over HTTPS (T1071.001)
- **Persistence** — Registry Run Key auto-start of the beacon (T1547.001)
- **Defense Evasion** — Base64 payload concealment and runtime decode-and-execute (T1027, T1140)

Case Index

#	CASE STUDY	ATT&CK	KEY TELEMETRY	PEAK SEVERITY
01	Suspicious PowerShell / Payload Execution – featured	T1059.001, T1059.003, T1074.001, T1071.001	Sysmon EID 1, EID 3, EID 11	HIGH
02	Registry Run Key Persistence	T1547.001	Sysmon EID 13	HIGH
03	C2 / Beaconing	T1071.001, T1059.001	Sysmon EID 1, EID 3, EID 22	HIGH

All three case studies below share the same simulated host (C:\USB_Test\RECYCLER.BIN) and represent a single continuous, chronologically ordered intrusion chain rather than isolated incidents.

Reading Guide

Each featured case study below follows the same structure so findings can be compared side by side:

- Executive summary & hunt hypothesis
- Attack flow and ATT&CK technique mapping
- Simulated payload / script evidence
- Screenshot evidence from Sysmon & Event Viewer
- KQL detection logic mapped to the observed telemetry
- Severity-rated detection opportunities
- Hunt result and analyst assessment

Scope Note

The full lab spans five stages (USB Execution, Discovery/Staging, Beaconing/C2, Registry Persistence, and Payload Obfuscation). This compendium features the three case studies most relevant to detection engineering review, in the order below.

CASE STUDY 01

FEATURED

Suspicious PowerShell / Payload Execution

Host path: C:\USB_Test\RECYCLER.BIN\ | Parent: Removable Disk (57GB).bat |

Telemetry: Sysmon EID 1 / 3 / 11

T1059.001 · PowerShell

T1059.003 · Windows Command Shell

T1074.001 · Local Data Staging

T1071.001 · Web Protocols

T1036 · Masquerading

Executive Summary

Following execution from a simulated removable-media staging path, a batch file masquerading as removable-media firmware spawned command interpreters and a PowerShell process that staged discovery output locally and triggered outbound web traffic. This case study validates end-to-end visibility into the moment a suspicious payload transitions from execution into active PowerShell-driven data staging — the pivot point most SOC playbooks rely on for early containment.

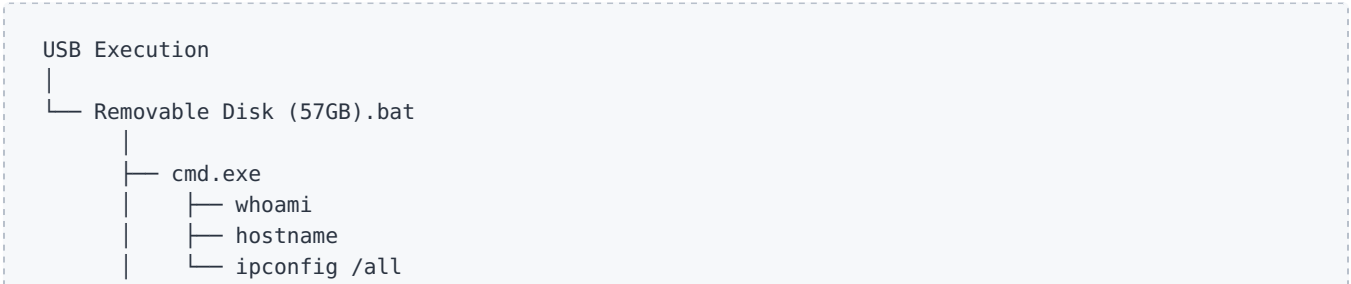
Hunt Hypothesis

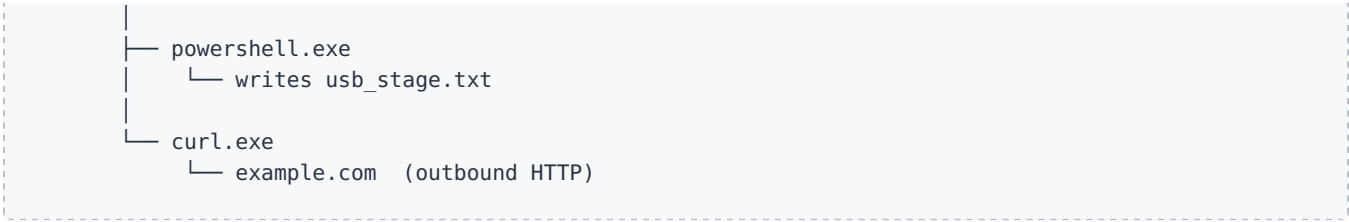
An adversary executing a payload from a removable-media-like location will immediately spawn command interpreters to fingerprint the host, stage the results locally through PowerShell, and generate outbound traffic to test connectivity or begin data transmission — all within seconds of the initial execution event.

Lab Environment

COMPONENT	VALUE
Operating System	Windows 11
Telemetry	Sysmon
Investigation Tool	Event Viewer
Campaign Inspiration	TEMP.Hex / SOGU

Attack Flow





Simulated Payload

```
@echo off
title Removable Disk
echo Simulating USB execution...

cmd.exe /c whoami > %TEMP%\usb_user.txt
cmd.exe /c hostname >> %TEMP%\usb_user.txt
cmd.exe /c ipconfig /all >> %TEMP%\usb_network.txt

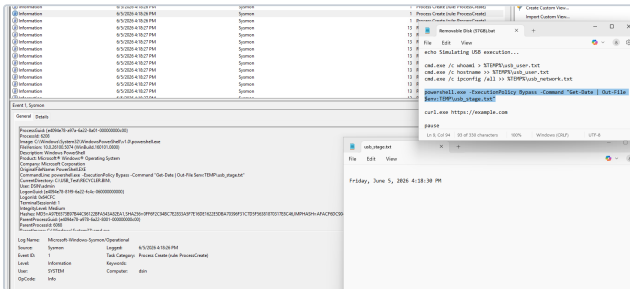
powershell.exe -ExecutionPolicy Bypass -Command "Get-Date | Out-File $env:TEMP\usb_stage.txt"

curl.exe https://example.com
pause
```

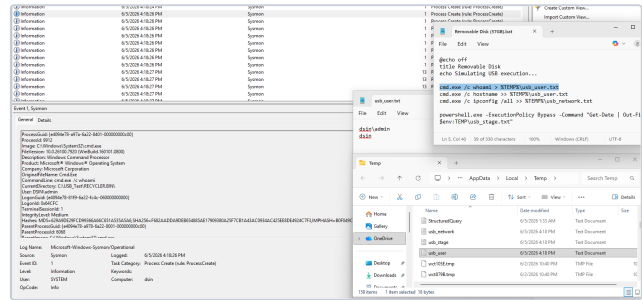
Evidence Collected

EVIDENCE	OBSERVATION
Parent Process	Removable Disk (57GB).bat
Child Process	cmd.exe, powershell.exe, curl.exe
Discovery Commands	whoami, hostname, ipconfig /all
Staging Artifacts	usb_user.txt, usb_network.txt, usb_stage.txt (%TEMP%)
Network Activity	outbound HTTP request to example.com
PowerShell Flag	-ExecutionPolicy Bypass

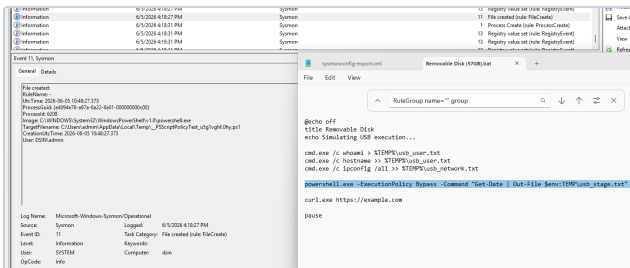
Screenshot Evidence



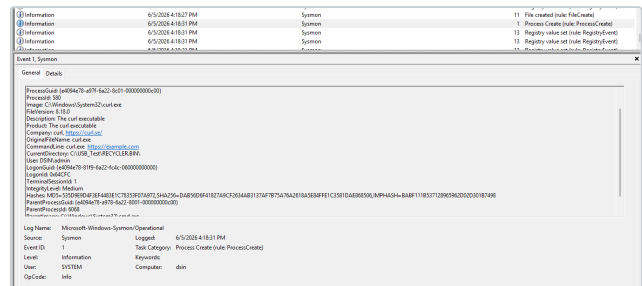
Sysmon EID 1 – PowerShell staging execution (usb_stage.txt)



Sysmon EID 1 – whoami / hostname / ipconfig discovery chain



Sysmon EID 11 – local staging file creation in %TEMP%



Sysmon EID 3 – outbound HTTP connection to example.com

KQL Detection Logic

Queries below are written against Microsoft Defender advanced hunting tables and mapped directly to the evidence chain observed in this hunt.

```
// Detection 1 -- Process execution from a removable-media-like staging path
DeviceProcessEvents
| where FolderPath has_any ("RECYCLER.BIN", "$Recycle.Bin", "System Volume Information")
| project Timestamp, DeviceName, FileName, FolderPath, ProcessCommandLine,
InitiatingProcessFileName
```

```
// Detection 2 -- Discovery command chain spawned within seconds of a suspicious parent
DeviceProcessEvents
| where InitiatingProcessFolderPath has "RECYCLER.BIN"
| where FileName in~ ("whoami.exe", "hostname.exe", "ipconfig.exe", "cmd.exe", "powershell.exe")
| summarize Commands = make_set(FileName), FirstSeen = min(Timestamp), LastSeen = max(Timestamp)
by DeviceName, InitiatingProcessId
| extend WindowSeconds = datetime_diff('second', LastSeen, FirstSeen)
| where WindowSeconds <= 30
```

```
// Detection 3 -- PowerShell staging output immediately followed by outbound traffic
let staging = DeviceFileEvents
| where FolderPath has "Temp" and FileName has "usb_"
| project StageTime = Timestamp, DeviceName, InitiatingProcessId;
DeviceNetworkEvents
| where RemoteUrl has "example.com" or InitiatingProcessFileName == "curl.exe"
| join kind=inner staging on DeviceName, $left.InitiatingProcessId == $right.InitiatingProcessId
```

```
| where Timestamp - StageTime between (0s .. 60s)
| project Timestamp, DeviceName, RemoteUrl, InitiatingProcessFileName, StageTime
```

Detection Opportunities

#	DETECTION LOGIC	SEVERITY
1	Execution originating from a RECYCLER.BIN-style staging path	MEDIUM
2	PowerShell launched with -ExecutionPolicy Bypass	MEDIUM
3	Discovery commands (whoami/hostname/ipconfig) executed immediately after suspicious execution	MEDIUM
4	Suspicious execution followed by outbound network communication within the same process tree	HIGH

Hunt Result

The hunt confirmed full telemetry coverage across command execution, discovery, local staging, and outbound communication. Sysmon Event IDs 1, 3, and 11 together reconstruct the entire execution-to-egress chain, giving defenders a reliable, low-noise signal set for this stage of a TEMP.Hex-style intrusion.

Analyst Assessment

While every command in this simulation was benign, the process lineage — a masquerading batch file spawning cmd.exe and powershell.exe, staging output locally, then generating outbound traffic within seconds — is structurally identical to early-stage malware execution and hands-on-keyboard reconnaissance observed in real TEMP.Hex/SOGU activity. This sequence should be treated as a high-value pivot point for containment.

CASE STUDY 02

Registry Run Key Persistence

Registry path: HKCU\Software\Microsoft\Windows\CurrentVersion\Run | **Value:**

WindowsUpdateService | **Telemetry:** Sysmon EID 13

T1547.001 · Registry Run Keys / Startup Folder

T1059.001 · PowerShell

T1071.001 · Web Protocols

T1036 · Masquerading

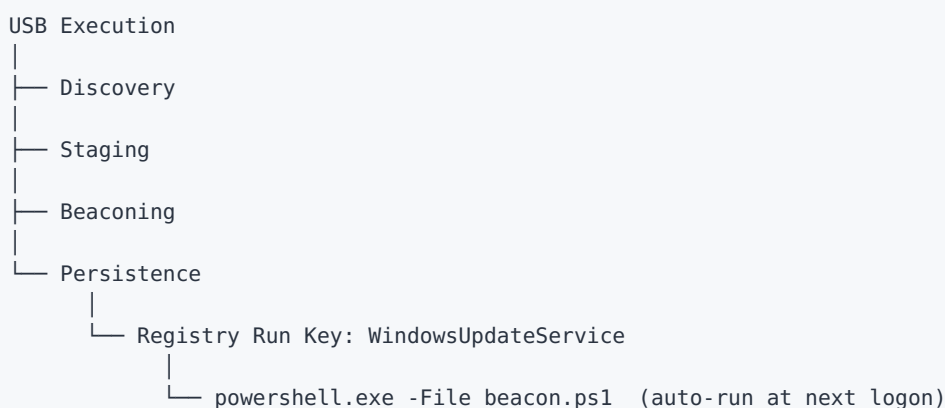
Executive Summary

With execution, discovery, staging, and beaconing already established, the campaign's final phase created durable persistence. A PowerShell one-liner wrote a Registry Run Key disguised as a legitimate Windows service ("WindowsUpdateService") that automatically relaunches the beacon script (beacon.ps1) at every user login. This case study validates visibility into Registry-based persistence via Sysmon Event ID 13 and independent registry inspection.

Hunt Hypothesis

An adversary establishing Registry Run Key persistence will do so via a scripting engine (PowerShell) shortly after C2 is established, using a value name that masquerades as a legitimate OS component to blend in with normal Run key contents.

Attack Flow



Persistence Script

```
$RunPath = "HKCU:\Software\Microsoft\Windows\CurrentVersion\Run"

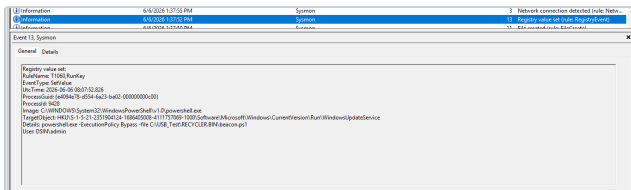
New-ItemProperty `
    -Path $RunPath `
```

```
-Name "WindowsUpdateService" `
-Value "powershell.exe -ExecutionPolicy Bypass -File C:\USB_Test\RECYCLER.BIN\beacon.ps1" `
-PropertyType String `
-Force
```

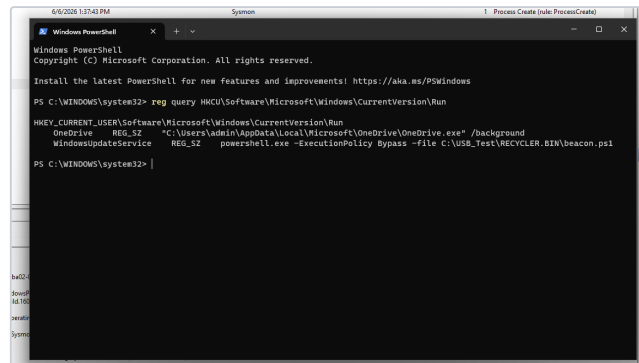
Evidence Collected

EVIDENCE	OBSERVATION
Event ID	13 (Sysmon – Registry Value Set)
Process	powershell.exe
Registry Path	HKCU\Software\Microsoft\Windows\CurrentVersion\Run
Registry Value	WindowsUpdateService
Configured Command	powershell.exe -ExecutionPolicy Bypass -File C:\USB_Test\RECYCLER.BIN\beacon.ps1
Persistence Status	Verified via reg query

Screenshot Evidence



Sysmon EID 13 – Registry Run Key creation (WindowsUpdateService)



Manual verification via reg query confirming the persisted value

KQL Detection Logic

```
// Detection 1 -- PowerShell writing to a Run / RunOnce Registry key
DeviceRegistryEvents
| where ActionType == "RegistryValueSet"
| where RegistryKey has_any (
    "CurrentVersion\Run", "CurrentVersion\RunOnce")
| where InitiatingProcessFileName =~ "powershell.exe"
| project Timestamp, DeviceName, RegistryKey, RegistryValueName,
    RegistryValueData, InitiatingProcessCommandLine
```

```
// Detection 2 -- ExecutionPolicy Bypass referenced inside a persisted Run value
DeviceRegistryEvents
```

```
| where ActionType == "RegistryValueSet"
| where RegistryKey has "CurrentVersion\Run"
| where RegistryValueData has "-ExecutionPolicy Bypass"
| project Timestamp, DeviceName, RegistryValueName, RegistryValueData
```

Detection Opportunities

#	DETECTION LOGIC	SEVERITY
1	Registry Run Key creation under CurrentVersion\Run / RunOnce	HIGH
2	PowerShell process creating Registry-based persistence	HIGH
3	-ExecutionPolicy Bypass present in the command used to create persistence	MEDIUM
4	Registry persistence correlated with prior beaconing activity from the same host	HIGH

Hunt Result

Sysmon Event ID 13 fully captured the Registry Value Set operation, and independent reg query inspection corroborated the finding. The combination provides two independent confirmation sources for Registry-based persistence, closing the loop on the simulated intrusion lifecycle.

Analyst Assessment

The value name "WindowsUpdateService" is a deliberate masquerade (T1036) designed to blend into normal Run key contents. Analysts triaging Run key telemetry should treat unfamiliar or update-themed value names that launch powershell.exe with -ExecutionPolicy Bypass as high-confidence persistence indicators, particularly when they correlate with prior beaconing activity on the same host.

CASE STUDY 03

C2 / Beaconing

Script: beacon.ps1 | **Destination:** example.com : 443 | **Telemetry:**

Sysmon EID 1 / 3 / 22

T1071.001 · Application Layer Protocol: Web Protocols

T1059.001 · PowerShell

T1036 · Masquerading

T1074.001 · Local Data Staging

T1105 · Ingress Tool Transfer (Simulated)

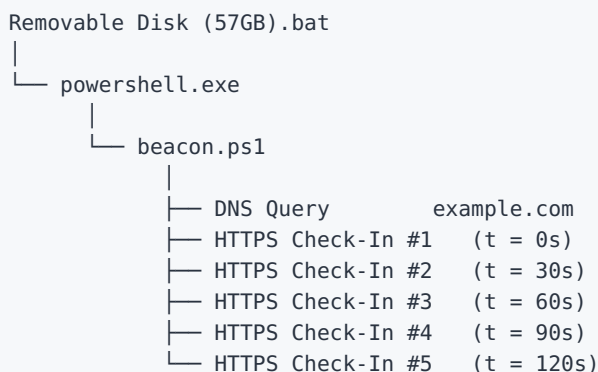
Executive Summary

Building on the discovery and staging activity established earlier in the chain, a secondary PowerShell script (beacon.ps1) was launched from the same removable-media-style staging directory and configured to generate five recurring outbound HTTPS requests at a fixed 30-second interval. This case study validates visibility into DNS activity, process lineage, and — most importantly — the fixed-interval periodicity that distinguishes beaconing from ordinary user-driven browsing.

Hunt Hypothesis

Command-and-control beacons implemented with a hard-coded sleep interval will produce outbound requests to the same destination at near-constant intervals with little to no jitter, which is statistically distinguishable from organic network traffic.

Attack Flow



Beacon Script

```
$Log = "$env:TEMP\beacon_log.txt"

1..5 | ForEach-Object {
    "Beacon Check-In: $(Get-Date)" | Out-File $Log -Append
}
```

```
Invoke-WebRequest -Uri "https://example.com" -UseBasicParsing | Out-Null

Start-Sleep -Seconds 30
}
```

Periodicity Analysis

A fixed `Start-Sleep -Seconds 30` with no randomized jitter produces a near-perfectly uniform check-in cadence — the single strongest statistical signal available for this hunt.

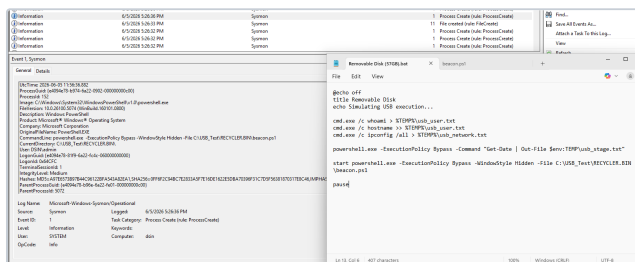
CHECK-IN #	ΔT FROM PREVIOUS	CUMULATIVE ELAPSED	DESTINATION
1	—	t + 0s	example.com : 443
2	30s	t + 30s	example.com : 443
3	30s	t + 60s	example.com : 443
4	30s	t + 90s	example.com : 443
5	30s	t + 120s	example.com : 443

Zero jitter ($\sigma \approx 0s$) across five consecutive intervals is itself a detection signal: legitimate application traffic almost never repeats at a perfectly constant interval over multiple cycles.

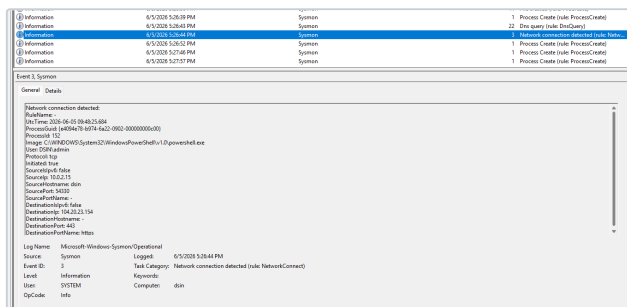
Evidence Collected

EVIDENCE	OBSERVATION
Parent Process	Removable Disk (57GB).bat
Child Process	powershell.exe → beacon.ps1
DNS Activity	example.com
Network Activity	HTTPS, destination port 443
Local Artifact	beacon_log.txt
Check-In Count	5, at fixed 30-second intervals

Screenshot Evidence



Sysmon EID 1 – PowerShell launching beacon.ps1 from RECYCLER.BIN



Sysmon EID 3 – outbound HTTPS connection generated by the beacon process

KQL Detection Logic

```
// Detection 1 -- PowerShell launched from a removable-media-like staging path
DeviceProcessEvents
| where FolderPath has "RECYCLER.BIN"
| where FileName =~ "powershell.exe"
| project Timestamp, DeviceName, ProcessCommandLine, InitiatingProcessFileName
```

```
// Detection 2 -- Fixed-interval beaconing: repeated connections to one remote host
DeviceNetworkEvents
| where InitiatingProcessFileName =~ "powershell.exe"
| where RemoteUrl has "example.com" or RemotePort == 443
| summarize Checkins = count(), Times = make_list(Timestamp)
    by DeviceName, InitiatingProcessId, RemoteUrl
| where Checkins >= 4
| extend Deltas = series_subtract(
    array_shift_left(Times, -1), Times)
```

```
// Detection 3 -- DNS query immediately followed by HTTPS connection to the same host
DeviceEvents
| where ActionType == "DnsQueryResponse" and AdditionalFields has "example.com"
| join kind=inner (
    DeviceNetworkEvents | where RemotePort == 443
) on DeviceName
| where NetworkTimestamp - Timestamp between (0s .. 5s)
```

Detection Opportunities

#	DETECTION LOGIC	SEVERITY
1	PowerShell launched from RECYCLER.BIN	HIGH
2	-ExecutionPolicy Bypass observed on the beacon process	MEDIUM
3	DNS query immediately followed by an HTTPS connection to the same host	MEDIUM

4	Repeated, fixed-interval outbound connections from the same process (low jitter)	HIGH
5	PowerShell script execution from a removable-media-like staging directory	HIGH

Hunt Result

Sysmon telemetry (process creation, DNS, and network connection events) fully reconstructed the beacon lifecycle from launch through five check-ins. The near-zero jitter across all five intervals provides a high-confidence, low-false-positive detection signal that generalizes well to real-world fixed-sleep C2 frameworks.

Analyst Assessment

This stage marks the clearest transition from "suspicious execution" to "active command-and-control" in the simulated chain. The combination of a masquerading parent process, PowerShell execution from a staging directory, and constant-interval HTTPS check-ins is a textbook beaconing signature. In production, this detection logic should be tuned with an allow-list for legitimate scheduled-task or update-checker traffic to control false positives, while treating destination novelty (rare/newly-observed domains) as an additional confidence multiplier.